**PYTHON PROGRAMMING**

**CASE STUDY ASSIGNMENT – 9**

# Case Study 9 – Smart Canteen Order and Stock Management

## 1. Problem Statement

**Managing food orders and available stock in a college canteen can be challenging, especially during busy hours. Canteen staff need to calculate order totals, check item availability, and ensure that the requested quantities do not exceed the available stock.**

**The Smart Canteen Order and Stock Management system is a Python-based program that allows users to select food items, enter quantities, calculate the total bill, and update the remaining stock. The program checks stock availability before accepting an order and displays the final bill and updated inventory. This system helps improve order processing, reduce stock errors, and simplify basic canteen management.**

**2. Objectives**

1. **To display available food items, their prices, and stock quantities.**
2. **To allow users to select a food item and enter the required quantity.**
3. **To calculate the total cost of each order.**
4. **To check whether the requested quantity is available in stock.**
5. **To update the stock quantity after a successful order.**
6. **To display the final bill and remaining stock.**
7. **To prevent invalid orders caused by unavailable items or invalid quantities.**
8. **To apply Python concepts such as dictionaries, loops, conditional statements, and arithmetic operations to a practical canteen management problem.**

**3. Assumptions**

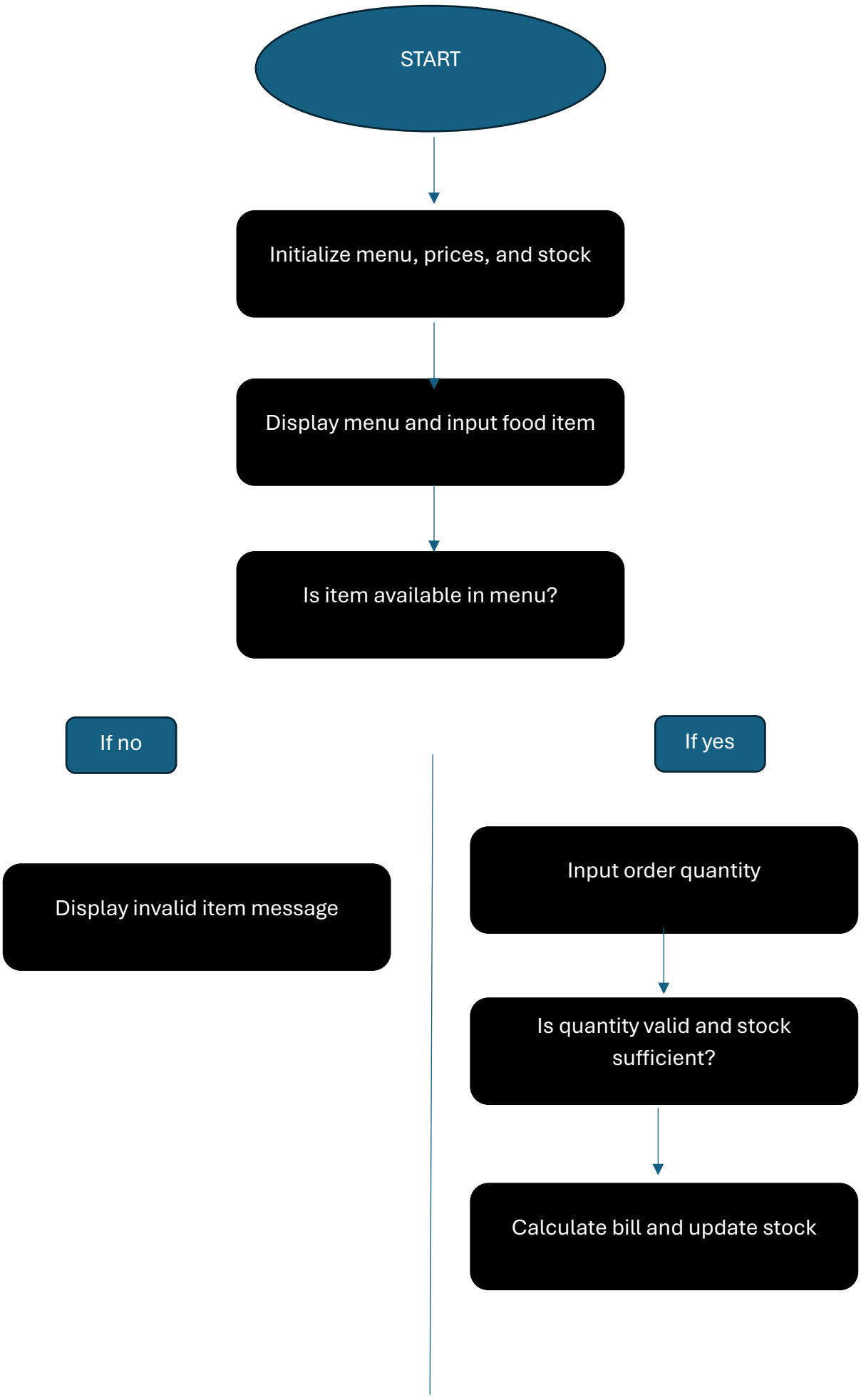
1. **The canteen offers a fixed menu of food items.**
2. **Each food item has a fixed price and an available stock quantity.**

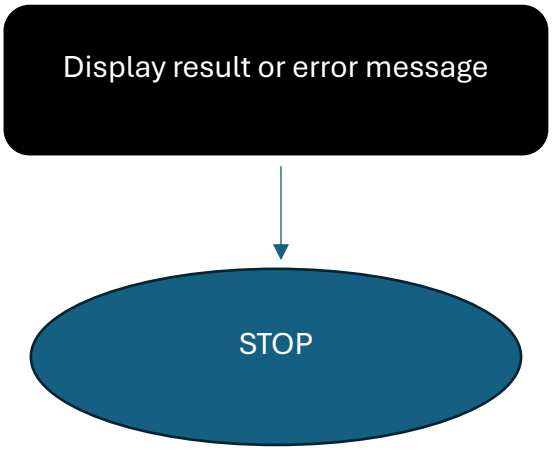
3. **The customer selects food items using their names from the displayed menu.**
4. **The order quantity must be a positive integer.**
5. **An order is accepted only if the requested quantity does not exceed the available stock.**
6. **The total cost is calculated by multiplying the item price by the ordered quantity.**
7. **Stock is reduced only after a valid order is accepted.**
8. **The program processes one order at a time.**
9. **Discounts, taxes, and delivery charges are not included in the bill.**
10. **Item names are entered using the names displayed in the menu.**

## 4. Algorithm

1. **Start the program.**
2. **Initialize a menu containing food item names, prices, and stock quantities.**
3. **Display the available menu.**
4. **Input the customer's selected food item.**
5. **Check whether the selected item exists in the menu.**
6. **If the item does not exist, display an invalid item message.**
7. **Otherwise, input the required quantity.**
8. **Check whether the quantity is a positive integer.**
9. **If the quantity is invalid, display an error message.**
10. **Otherwise, check whether the requested quantity is available in stock.**
11. **If sufficient stock is available:**
    - **Calculate the total cost using price multiplied by quantity.**
    - **Deduct the ordered quantity from the available stock.**
    - **Display the order details, total bill, and remaining stock.**
12. **If sufficient stock is not available, display an insufficient stock message.**
13. **Stop the program.**

## 5.Flowchart


START
Initialize menu, prices, and stock
Display menu and input food item
Is item available in menu?
If no
Display invalid item message
If yes
Input order quantity
Is quantity valid and stock sufficient?
Calculate bill and update stock




## 6.psuedo code

**BEGIN**

**Initialize menu with food items, prices, and stock quantities**

**Display available food items**

**INPUT item_name**

**IF item_name is not present in menu THEN**

**DISPLAY "Invalid food item"**

**ELSE IF quantity > available stock THEN**

**DISPLAY "Insufficient stock"**

**ELSE**

**total_bill = price × quantity**

**available_stock = available_stock - quantity**

**DISPLAY "Order placed successfully"**

DISPLAY item_name

DISPLAY quantity

DISPLAY total_bill

DISPLAY remaining stock

END IF

END IF

END

## 7.Source code

Smart Canteen Order and Stock Management.py

PYTHON-ACTIVITY-BASED-LEARNING > Smart Canteen Order and Stock Management.py > ...

```python

menu = {
    "Burger": {"price": 80, "stock": 10},
    "Sandwich": {"price": 50, "stock": 8},
    "Pizza": {"price": 120, "stock": 5},
    "Juice": {"price": 30, "stock": 12}
}

print("SMART CANTEEN ORDER AND STOCK MANAGEMENT")
print("\nAvailable Menu:")
for item, details in menu.items():
    print(item, "- Price: Rs.", details["price"],
          "| Stock:", details["stock"])

item_name = input("\nEnter food item: ").title()

if item_name not in menu:
    print("Invalid food item")
else:
    quantity = int(input("Enter quantity: "))

    if quantity <= 0:
        print("Invalid quantity")

    elif quantity > menu[item_name]["stock"]:
        print("Insufficient stock")

    else:
        total_bill = menu[item_name]["price"] * quantity

        menu[item_name]["stock"] -= quantity

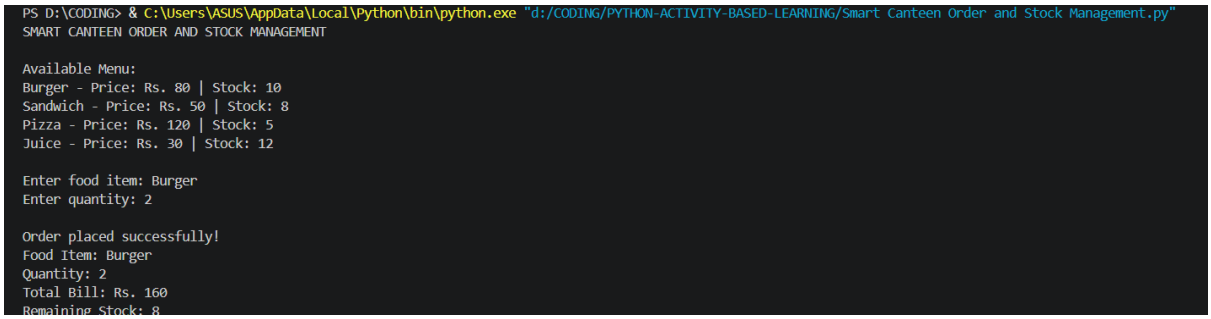
        print("\nOrder placed successfully!")
        print("Food Item:", item_name)
        print("Quantity:", quantity)
        print("Total Bill: Rs.", total_bill)
        print("Remaining Stock:",
              menu[item_name]["stock"])
```

## 8. Test Cases

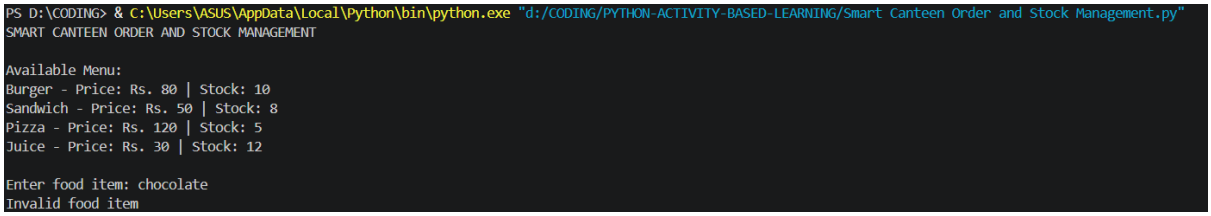
| Test Case | Food Item | Quantity | Expected Output |
|---|---|---|---|
| 1 | Burger | 2 | Order successful, bill ₹160, remaining stock 8 |
| 2 | Sandwich | 1 | Order successful, bill ₹50, remaining stock 7 |
| 3 | Pizza | 2 | Order successful, bill ₹240, remaining stock 3 |
| 4 | Juice | 0 | Invalid quantity |
| 5 | Chocolate | 1 | Invalid food item |
| 6 | Burger | 15 | Insufficient stock |

## 9. Output Screenshots

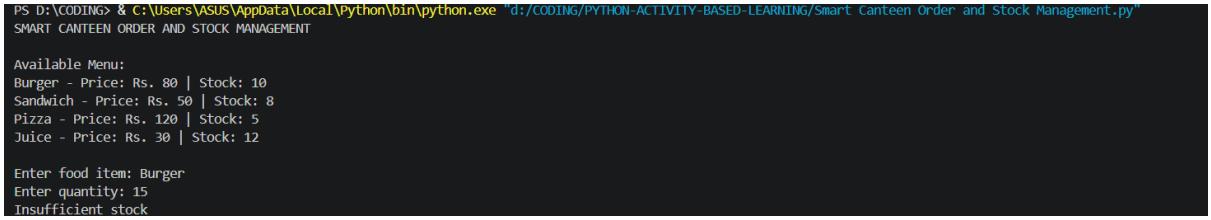
### Screenshot 1: Successful Order




### Screenshot 2: Invalid Food Item



## Screenshot 3: Insufficient Stock




## 10. Conclusion

**The Smart Canteen Order and Stock Management system was successfully developed using Python. The program allows users to select food items, enter quantities, calculate the total bill, and check stock availability. It updates the remaining stock after every successful order and handles invalid food items, invalid quantities, and insufficient stock. This project demonstrates the practical use of dictionaries, loops, conditional statements, user input, and arithmetic operations in Python.**